GatePass — Solution

Category: WEB | Points: 650 | Author: Jerit3787

Overview

A JSON parser differential. A Go gateway authorizes requests with `gjson` and forwards the raw bytes to a Python backend that re-parses with `json.loads`. The two libraries disagree twice, and both disagreements are needed to reach the flag:

1. On duplicate keys, `gjson` keeps the FIRST value; `json.loads` keeps the LAST.
2. On a dotted key like `user.clearance`, `gjson` interprets it as a NESTED path (and cannot see a literal top-level `"user.clearance"` key); `json.loads` reads it as a literal key.

Steps

1. Probe the gateway (no source is shipped): `role: admin` is refused ("role escalation blocked"), `clearance > 3` is refused ("clearance too high"), and so is a nested `user.clearance`. `/api/whoami` reports the role and effective clearance the backend derived.
2. **Stage 1 (duplicate-key):** `POST /api/whoami` with `{"role":"user","role":"admin"}` — the guard reads the first `role` (`user`) and forwards; the backend reads the last (`admin`). The response confirms it.
3. `POST /api/vault` with that body -> `clearance too low (required 9000)`. You now need `user.clearance >= 9000`, but the guard caps it, and a nested `user.clearance` is caught (and ignored by the backend anyway).
4. **Stage 2 (dotted-key):** put clearance under a LITERAL dotted key. `gjson.Get( "user.clearance")` looks for a nested object and finds nothing (invisible to the guard), while the backend reads the literal key:

   ```
   {"role":"user","role":"admin","user.clearance":9999}
   ```

5. `POST /api/vault` with that body -> the flag.

See `solve.py` for the automated chain (send raw bytes so the duplicate keys survive).

Red Herrings

- `role: "admin"` / `clearance: 9999` plainly -> 403 (the guard sees them).
- Nested `{"user":{"clearance":9999}}` -> caught by the guard's path lookup, and the backend's literal-key reader ignores it anyway.
- Duplicate-key on `clearance` -> doesn't help: the backend derives clearance only from the literal `user.clearance` key, not from the top-level `clearance`.

Flag

```
gctf26{tw0_p4rs3rs_w4lk_1nt0_4_b4r_0n3_l34v3s}
```